

Who Goes Next

1 Overview

A hospital waiting room calls in one patient every two minutes. Two policies are compared over one simulated shift of 20,000 arrivals (10,000 patients called): **No triage** — first come, first served, severity ignored. **Triage** — the most severe patient waiting is always called next; among equally severe patients, whoever arrived first goes first.

You implement four data structures, each a different way of deciding who goes next. Three are yours to write from scratch; one (ArrayRoom) is given as a trusted reference.

Class	Structure	Policy	Who writes it
ArrayRoom	dynamic array	triage	given — trusted reference, read-only
FifoQueueRoom	linked queue	no triage	you — Task 1
SortedListRoom	sorted linked list	triage	you — Task 2
HeapRoom	binary max-heap	triage	you — Task 3
CallLog	array-backed stack	records every call	you — Task 4

FifoQueueRoom, SortedListRoom and HeapRoom implement the same WaitingRoom interface (add, next, size, isEmpty) and run through the exact same Shift/Main. Never use `java.util.ArrayList`, `LinkedList`, `ArrayDeque`, `PriorityQueue`, `Stack` or `Collections.sort` in the four classes you write — the point is the structures themselves, built by hand.

2 Starting point

Given, read-only: Patient.java, WaitingRoom.java, ArrayRoom.java, Shift.java, Main.java, Bench.java, patients.csv.

You add: FifoQueueRoom.java, SortedListRoom.java, HeapRoom.java, CallLog.java. Nothing else changes.

3 Your personal dataset

patients.csv has 60,000 rows. Your barcode selects a 20,000-row window of it automatically:

```
java Main <your barcode>
```

Two students almost never get the same window, so two submissions with the same code will not produce the same numbers. This is what makes copy-pasted code visible, and what lets the checks below confirm your own code is right without anyone comparing submissions by hand.

4 The four tasks

TASK 1 • THE WARM-UP

FifoQueueRoom — queue

A singly linked queue: Node with a `next` pointer, a `head` and a `tail`. `add` links onto the tail; `next` unlinks the head. Severity is ignored completely.

Because it ignores severity, a correct `FifoQueueRoom` always produces the same checksum, on **any** barcode: $1^2 + 2^2 + \dots + 10000^2$. `Main` checks this for you — no comparison against anyone else's code needed.

TASK 2

SortedListRoom — list

A singly linked list kept in call order at all times, so `next` is just "unlink the head." `add` walks from the head to where the new patient belongs.

More severe first; among equal severities, earlier arrival (smaller id) goes first — ask `Patient.before`, never compare severities yourself. **Common bug:** stopping one node too early on ties, which flips their order.

`add` worst-case $\Theta(n)$ · `next` $\Theta(1)$

TASK 3 • THE HARDEST

HeapRoom — binary max-heap

A max-heap in a plain array — no node objects. Index `i`: parent $(i-1)/2$, children $2i+1$, $2i+2$. `add` appends and sifts up; `next` removes the root, moves the last element in, and sifts down.

Common bug: `siftDown` must compare **both** children, not just one — a short hand-trace often won't catch this; it takes a full shift to expose.

`add/next` $O(\log n)$ amortised · $\Theta(n)$ worst case (the resize)

TASK 4

CallLog — stack

A stack backed by your own growing array (double it when full). Needs `push`, `pop`, `peek`, `size`, `isEmpty`, and `recent(n)`: the `n` most recently called patients, most recent first.

`recent(n)` must be **read-only** — same `size()`, same answer on a second call. Never implement it by popping. You never call `push` yourself; `Shift` does, once per `next()`.

5 Worked example: amortised vs worst case

`ArrayRoom.add` is given, but its complexity is the model for `HeapRoom.add` and `CallLog.push`, which resize the same way (doubling the backing array when full).

WORST CASE — ONE CALL

$\Theta(n)$

The one call that finds the array full allocates a new array of size $2n$ and copies all n elements before adding the new one.

AMORTISED — OVER N CALLS

$O(1)$ per call

A resize only happens every $2\times$ -further-away call, so total copying over n calls is $O(n)$ — $O(1)$ spread per call.

Both numbers are true at once and answer different questions. The report and defense expect **both**, for whichever of your classes resize.

6 Checking your own work

`java Main <your barcode>`

Your five rows, and OK / WRONG for every class you've written.

`java Main <your barcode> trace 6`

Your list and heap, side by side, after the first 6 arrivals.

OK Call order matches the given ArrayRoom exactly, and `size()/isEmpty()` are consistent with what was added/removed.

WRONG Tells you the exact call # where your sequence first diverges from ArrayRoom's — go straight to `trace 6` there and compare columns by hand.

-- Class not written yet — both commands are safe from minute one; run them after every method, not just at the end.

7 Benchmarking with Bench.java

`java Bench <your barcode>` times array, list and heap (not `FifoQueueRoom` — every operation is $O(1)$). It runs a warm-up first, keeps the best of three timed runs, and folds results into a running total so the JIT can't delete the "dead" code.

Table A — whole shift, 3 sizes

1,000 / 5,000 / 20,000 patients — shows how total time grows; a $\Theta(n)$ op shows a much steeper curve than $O(\log n)$.

Table B — add vs next isolated

1,000 of each, in a room already holding 19,000 — shows directly which single operation is expensive.

Numbers are machine-dependent — they won't match a classmate's. What should match: the **shape** — which room slows down as n grows, and which operation (not room) is expensive for array vs list.

8 Report — two pages maximum

A short PDF or Markdown file, using the provided `REPORT.md` template:

1. **Correctness** — full output of `java Main <your barcode>`, everything showing OK.
2. **Complexity** — worst-case and amortised time for `SortedListRoom.add` and `HeapRoom.add`, modelled on the worked example.
3. **Benchmark** — both Bench tables, with a note on whether measured growth matches the complexity analysis.
4. **Triage effect** — using your own "never called" numbers, what triage buys the most severe patients and costs the least severe.

Short, specific answers that reference your own numbers score higher than long ones that restate this handout.

9 Submission

Submit through the course LMS:

- `FifoQueueRoom.java`, `SortedListRoom.java`, `HeapRoom.java`, `CallLog.java`
- Your report (PDF or Markdown, two pages)
- The full output of `java Main <your barcode>` (pasted into the report, not a separate file)

Do not submit:

`Patient.java`, `WaitingRoom.java`, `ArrayRoom.java`, `Shift.java`, `Main.java`, `Bench.java`, `patients.csv` — already on file, unchanged. Submitting your own copies only slows grading down.

Late submissions follow the course's standard late policy — check the syllabus/LMS, not this handout.

10 Grading — Submission 50 + Defense 50

Part	Points
FifoQueueRoom correct	5
SortedListRoom correct	10
HeapRoom correct	10
CallLog correct	5
Report	20
Submission total	50
Defense	50

"Correct" means `java Main` shows OK for that class — call order, `size()/isEmpty()`, and the empty-room exception, not just a matching checksum.

Without a defense, 50/100 is the maximum possible grade, regardless of how correct or polished the code and report are. The defense checks that the code reflects your own understanding — not a formality on top of it.

11 The defense

- Know your own code — point at any line of any of your four classes and say what it does and why.
- Be ready to trace it by hand, on paper, for arrivals the instructor picks on the spot.
- Be ready for a small change on the spot — e.g. the tie-break rule, or a bug fix.
- Justify your complexity numbers — not just state them, but where they come from in your code.
- Explain why your numbers are yours — different barcode, different dataset.

12 Academic integrity

The personal-barcode dataset means copying a classmate's code produces checksums and call orders that don't match your barcode — `java Main` shows WRONG even if the file compiles and looks plausible.

Using an AI to generate the four classes will likely pass the automated checks, but will likely fail the defense — graded entirely on your own understanding. The barcode and the defense together are the actual integrity check.